

Rapport de Projet JAVA

YapChat

Application de Chat Vocal en Temps Réel

Réalisé par :

Hammouchi Insaf

Maazouz Doha

Rebbouh Fatima Ezzahra

Encadré par :

Baddi Youssef

Année universitaire : 2025-2026

1. Introduction

1.1 Contexte du projet

Avec l'évolution des réseaux informatiques et des technologies de communication, les applications de messagerie instantanée sont devenues essentielles. Le projet **YapChat** vise à développer une application permettant la communication **textuelle et vocale en temps réel** entre plusieurs utilisateurs connectés via un réseau TCP/IP, en utilisant une architecture client-serveur.

1.2 Objectifs du projet

Les principaux objectifs de ce projet sont :

- Permettre la communication textuelle en temps réel entre utilisateurs
- Implémenter l'envoi et la réception de messages vocaux
- Proposer une interface utilisateur simple et intuitive
- Assurer une communication réseau fiable et stable

2. Architecture générale du système

2.1 Vue d'ensemble

L'application YapChat repose sur une architecture **client-serveur**, où le serveur joue le rôle de point central de communication et les clients échangent des messages via celui-ci.

2.2 Technologies utilisées

- **Langage de programmation** : Java (JDK 8 ou supérieur)
- **Protocole réseau** : TCP/IP
- **Interface graphique** : Java Swing
- **Communication réseau** : Sockets Java
- **Gestion audio** : Java Sound API (InputStream / OutputStream)

3. Analyse et description des composants

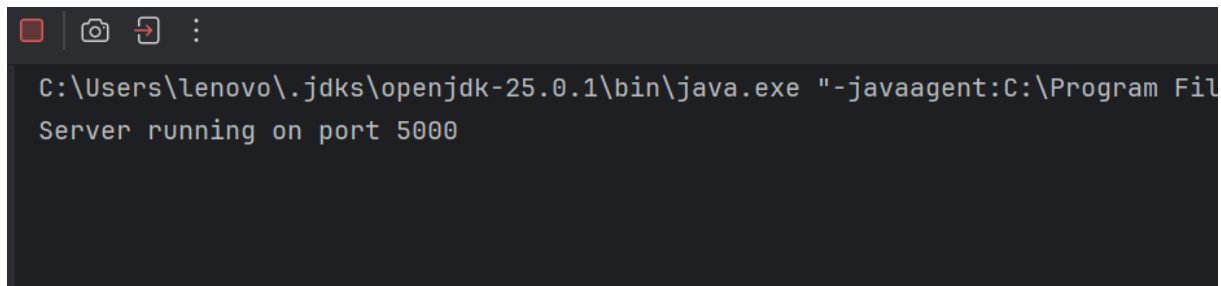
3.1 Serveur (Serveur.java)

Rôle :

Le serveur est le cœur de l'application. Il gère les connexions entrantes et assure la diffusion des messages entre les clients connectés.

Fonctionnalités principales :

- Écoute des connexions sur un port défini (5000)
- Gestion de plusieurs clients simultanément
- Diffusion des messages texte et audio
- Gestion de la déconnexion des clients



```
C:\Users\lenovo\.jdk\openjdk-25.0.1\bin\java.exe "-javaagent:C:\Program Files\Java\jdk-25.0.1\lib\jconsole.jar" -jar Server.jar
Server running on port 5000
```

3.2 Gestionnaire de clients (ClientHandler.java)

Rôle :

Chaque client connecté est pris en charge par un thread dédié.

Fonctionnalités :

- Réception des données envoyées par le client
- Transmission des messages au serveur pour diffusion
- Gestion propre de la déconnexion
-

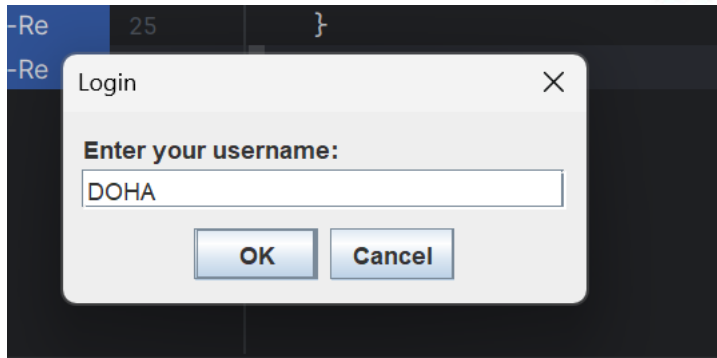
3.3 Client réseau (Client.java)

Rôle :

Assure la communication réseau entre l'application cliente et le serveur.

Fonctionnalités :

- Connexion au serveur
- Envoi des messages texte et audio
- Réception des messages diffusés
- Gestion des flux d'entrée et de sortie



3.4 Contrôleur réseau (NetworkController.java)

Rôle :

Intermédiaire entre l'interface graphique et la couche réseau.

Fonctionnalités :

- Gestion de la connexion et de la déconnexion
- Coordination de l'envoi et de la réception des messages
- Communication avec le module audio
- Notification de l'interface graphique

```

Client.java ClientHandler.java Audio.java NetworkController.java x ChatController.java
1 import java.io.IOException;
2
3 public class NetworkController implements ChatController { 4 usages 2 Doha-ma +1
4     private String username; 3 usages
5     private YapChatGUI gui; 8 usages
6     private Client client; 5 usages
7     private final Audio audioModule = new Audio(); 3 usages
8
9     // Constructor receives username
10    > public NetworkController(String username) { this.username = username; }
13
14    > public void setGUI(YapChatGUI gui) { this.gui = gui; }
17
18    // ===== CONNECTION =====
19    @Override 1 usage 2 Doha-ma +1
20    > public void onConnect(String host, int port) {
21        try {
22            this.client = new Client(host, port, controller: this); // pass controller to client
23            gui.enableChat();
24        } catch (IOException e) {
25            System.err.println("Erreur de connexion: " + e.getMessage());
26        }
27    }

```

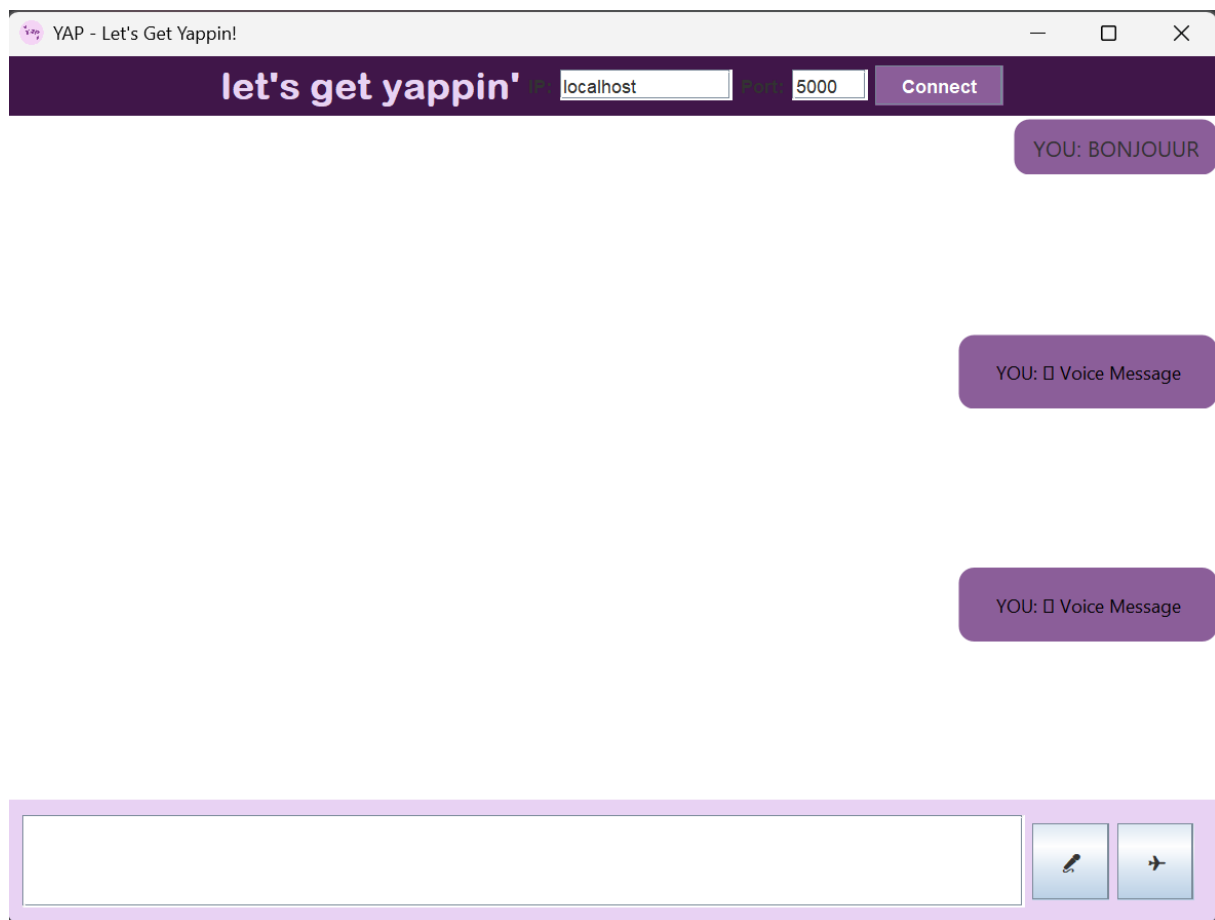
3.5 Interface utilisateur (YapChatGUI.java)

Rôle :

Permet à l'utilisateur d'interagir avec l'application.

Fonctionnalités principales :

- Saisie et affichage des messages texte
- Enregistrement et lecture des messages audio
- Gestion de la connexion au serveur



3.6 Point d'entrée de l'application (Main.java)

Rôle :

Initialise et lance l'application YapChat.

Fonctionnalités :

- Demande du nom d'utilisateur
- Initialisation du contrôleur réseau
- Lancement de l'interface graphique

4. Protocole de communication

4.1 Types de messages

L'application distingue deux types de messages :

- **Messages texte**

Les messages texte sont des chaînes de caractères envoyées par un client au serveur. Le serveur se charge ensuite de diffuser le message à tous les autres clients connectés. Ces messages sont utilisés pour la communication écrite classique entre les utilisateurs.

- **Messages audio**

Les messages audio correspondent à des enregistrements vocaux réalisés par l'utilisateur via le microphone. Les données audio sont capturées sous forme de tableau d'octets (byte[]), puis transmises au serveur.

Chaque message contient l'expéditeur et le contenu (texte ou données audio).

Message texte envoyé par l'utilisateur local :

YOU: test

Message audio envoyé par l'utilisateur local :

YOU: □ Voice Message

Message texte reçu d'un autre utilisateur :

fatii: test

Message audio reçu d'un autre utilisateur :

fatii: □ Voice Message

4.2 Flux de communication

- Connexion du client au serveur
- Envoi du message depuis l'interface utilisateur
- Transmission via le contrôleur réseau
- Diffusion par le serveur vers les autres clients

5. Gestion de l'audio

5.1 Module audio

La classe **Audio** est le module responsable de la gestion du son dans l'application YapChat. Elle permet :

- D'enregistrer la voix de l'utilisateur via le **microphone**
- De stocker l'audio sous forme de **tableau d'octets (byte[])**
- De jouer les messages vocaux reçus via les **haut-parleurs**
- De gérer le **streaming audio en temps réel** pour les appels vocaux
- D'améliorer la qualité sonore (normalisation + compression)

5.2 Configuration audio

La classe définit le format audio utilisé :

- Fréquence : 44.1 kHz (qualité CD)
- 16 bits
- Mono
- Signé
- Little endian

Cela garantit une **qualité correcte et un format standard** compatible avec Java Sound.

```
import javax.sound.sampled.*;
import java.io.ByteArrayOutputStream;
import java.util.concurrent.atomic.AtomicBoolean;

public class Audio {

    // ===== CONFIGURATION =====
    private static final float SAMPLE_RATE = 44100.0f;
    private static final int SAMPLE_SIZE = 16;
    private static final int CHANNELS = 1;
    private static final boolean SIGNED = true;
    private static final boolean BIG_ENDIAN = false;

    private AudioFormat audioFormat;
    private AtomicBoolean isRecording = new AtomicBoolean(initialValue: false);
    private TargetDataLine microphone;
    private SourceDataLine speaker;
    private ByteArrayOutputStream audioBuffer;
```

5.3 Enregistrement audio

Méthodes principales :

➔ **startRecording()**

- Ouvre le microphone
- Démarre un thread
- Commence à lire les données audio
- Stocke le son dans `ByteArrayOutputStream`

Donc quand l'utilisateur appuie sur le bouton d'enregistrement → ça démarre ici.

➔ **stopRecording()**

- Arrête le microphone
- Ferme la ligne audio
- Récupère les données enregistrées
- Retourne les données audio sous forme `byte[]`
- Applique une **normalisation** du volume

C'est ce qui permet ensuite d'envoyer l'audio au serveur.

5.4 Lecture audio

➔ **playAudio(byte[] audioData)**

Permet de lire un message vocal reçu :

- Ouvre les haut-parleurs (`SourceDataLine`)
- Envoie les données audio
- Joue le son

Donc quand un autre utilisateur t'envoie un message vocal → cette fonction le joue.

5.5 Amélioration de l'audio

➔ **Normalisation (normalizeAudio)**

- Détecte le volume maximum
- Augmente le son s'il est trop faible
- Évite les saturations

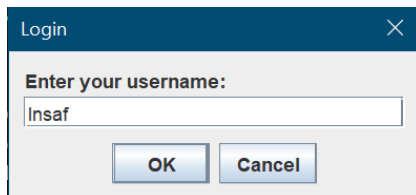
But → que tous les messages soient audibles.

➔ **Compression (compressAudio)**

- Réduit la taille audio
- Supprime certains échantillons
- Utile pour réseau lent

But → réduire la bande passante.

5.6 Test

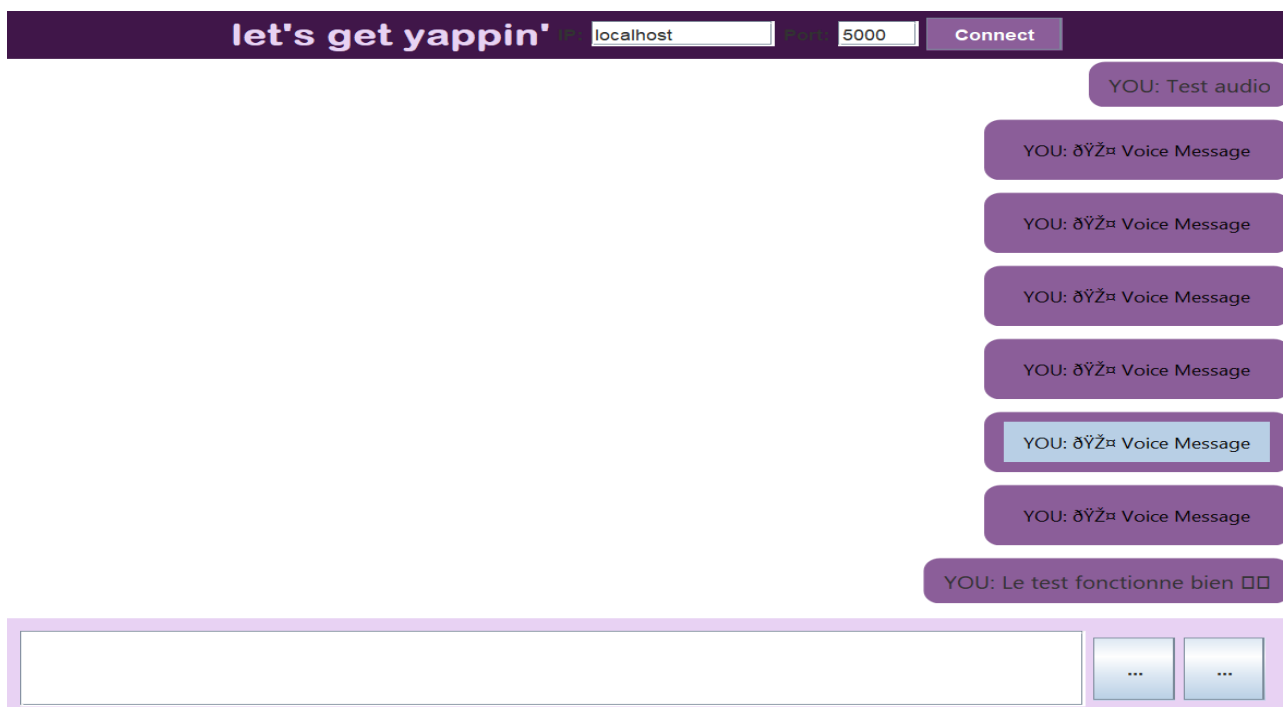


Login

Enter your username:

Insaf

OK Cancel



let's get yappin' localhost 5000 Connect

YOU: Test audio

YOU: ڤڤڤ Voice Message

YOU: ڤڤڤ Voice Message

YOU: ڤڤڤ Voice Message

YOU: ڤڤڤ Voice Message

YOU: ڤڤڤ Voice Message

YOU: ڤڤڤ Voice Message

YOU: Le test fonctionne bien ☐☐

...

...

5.6 Flux audio

- **Enregistrement** : Microphone → Données binaires → Réseau
- **Lecture** : Réseau → Données binaires → Haut-parleurs

6. Gestion des threads et de la concurrence

6.1 Côté serveur

- Thread principal pour l'écoute des connexions
- Un thread par client connecté

6.2 Côté client

- Thread principal pour l'interface graphique (Swing)
- Thread secondaire pour la réception des messages

6.3 Synchronisation

L'accès aux ressources partagées est synchronisé afin d'éviter les conflits entre threads.

7. Analyse critique du système

7.1 Points forts

- Architecture simple et claire
- Bonne séparation des responsabilités
- Support de la communication texte et audio
- Gestion multi-utilisateurs

7.2 Limites et améliorations possibles

- Absence de chiffrement des données
- Pas de système d'authentification
- Performances audio perfectibles
- Scalabilité limitée du serveur

8. Déploiement et tests

8.1 Prérequis

- Java JDK 8 ou supérieur
- Microphone et haut-parleurs

8.2 Lancement de l'application

- Démarrage du serveur
- Lancement de plusieurs clients

8.3 Tests réalisés

- Test de connexion multiple
- Envoi et réception de messages texte
- Envoi et réception de messages audio
- Test de déconnexion

9. Conclusion

Le projet YapChat met en œuvre une application fonctionnelle de chat en temps réel avec support audio. Il constitue une base solide pour le développement d'applications de communication plus avancées.

9.1 Résultats obtenus

- Communication client-serveur opérationnelle
- Support du texte et de l'audio
- Interface utilisateur conviviale

9.2 Perspectives d'évolution

- Ajout de salons de discussion
- Intégration de la vidéo
- Partage de fichiers
- Développement d'une version mobile ou web